

Juillet
2015

le cnam

NFE204 - Bases de données documentaires et distribuées

Rapport de projet

Mise en pratique des aspects abordés en cours à travers la récupération de données depuis une API REST, l'étude du système NoSQL RethinkDB, la synchronisation avec ElasticSearch, la réalisation d'un tableau de bord avec Kibana



Rémy MALICOT
Microsoft



Sommaire

Introduction	3
Découverte du système RethinkDB	4
Installation et démarrage	4
Découverte de l'interface web de RethinkDB	4
Premiers tests avec le langage de requête ReQL	5
Interrogation de l'API data.gouv.fr et insertion / mise à jour des documents dans RethinkDB	7
Présentation de l'API data.gouv.fr	7
Mise en œuvre du script de récupération et d'insertion / mise à jour des données	8
Pratique du langage de requête ReQL	10
Techniques de distribution	12
Etude du partitionnement et de la réplication	12
Mise en place d'un cluster et tests	13
Interfaçage avec le moteur de recherche Elasticsearch	15
Création d'un index avec mapping	16
Création d'une river	16
Réalisation d'un tableau de bord avec l'outil d'analyse Kibana	17
Création d'un index pattern	17
Présentation de l'interface	18
Réalisation d'un tableau de bord à partir des données	18
Conclusion	20

Introduction

Dans le cadre de la validation de l'UE NFE204 (Bases de données documentaires et distribuées), il nous a été demandé de réaliser un mini-projet, accompagné d'un rapport, portant sur les sujets vus en cours.

Ce projet est constitué d'une phase de récupération et chargement de données dans un système NoSQL de notre choix et d'une phase d'expérimentation.

Mon étude porte sur la récupération de données depuis l'API data.gouv.fr, l'insertion et la mise à jour des données dans le système NoSQL RethinkDB, la synchronisation automatique avec le moteur de recherche Elasticsearch et la réalisation d'un tableau de bord avec l'outil d'analyse Kibana.

Pour réaliser ce projet, j'ai utilisé les logiciels et composants suivants :

- RethinkDB (Version 2.0.1) : un système de gestion de bases de données NoSQL orienté documents
- Python (Version 2.7.9) : j'ai écrit un script en Python dans le but d'interroger l'API data.gouv.fr et d'alimenter / mettre à jour une base de données documentaire gérée par RethinkDB
- Driver client Python pour RethinkDB : ce driver va permettre la communication entre notre client et le serveur RethinkDB
- Elasticsearch (Version 1.5.2) : un moteur d'indexation et de recherche basé sur Lucene
- Elasticsearch RethinkDB River : un plugin permettant de synchroniser la base de données documentaire gérée par RethinkDB avec Elasticsearch (les mises à jour de la base de données sont reportées dans l'index). Pour que la river puisse fonctionner, il faut une version 1.3 ou 1.4 d'ElasticSearch et une version 8 de Java (Version 8.0 Update 45 installée dans mon cas)
- Kibana (Version 4.0.2) : un outil d'analyse permettant de réaliser des tableaux de bord à partir des données stockées dans l'index Elasticsearch

RethinkDB n'étant pas compatible Windows, j'ai également utilisé le système suivant pour le faire fonctionner :

- Oracle VM VirtualBox (Version 4.3.26) avec une VM Linux Debian 8.0

Découverte du système RethinkDB

Installation et démarrage

L'installation est réalisée en suivant les instructions indiquées sur le site de l'éditeur :

<http://rethinkdb.com/docs/install/debian>

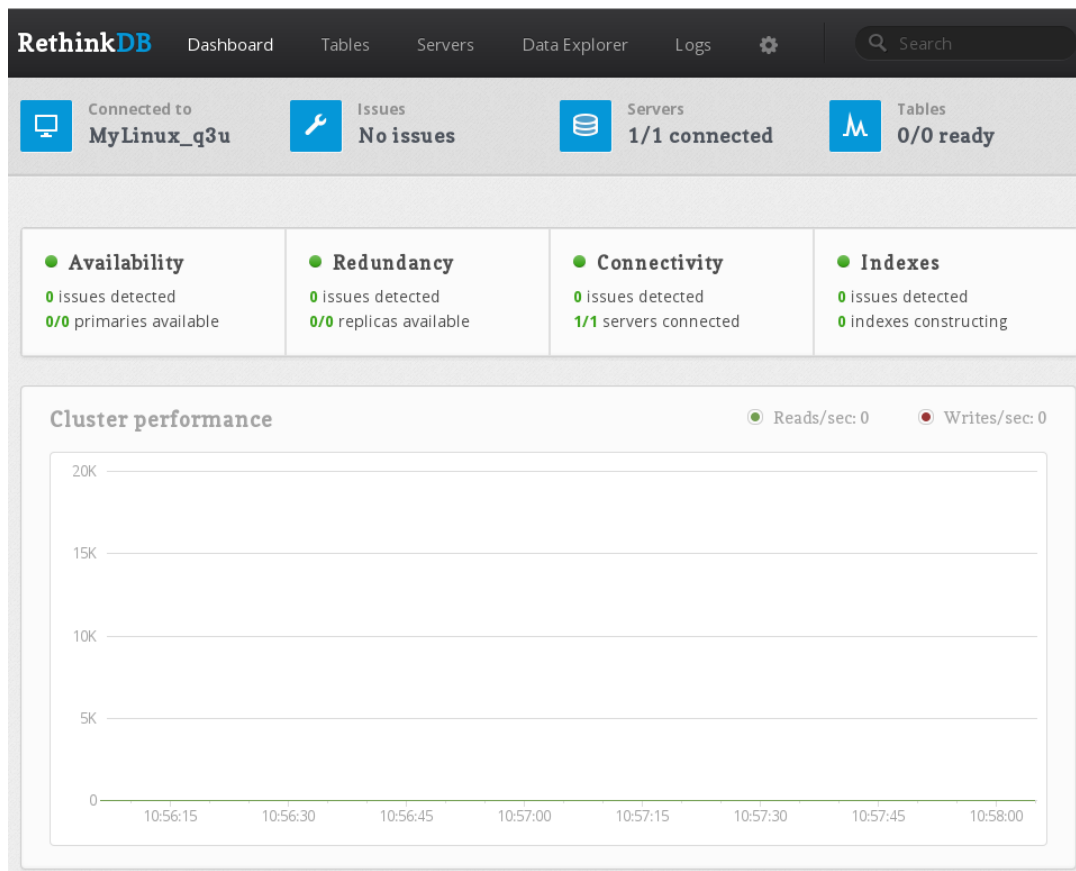
Une fois l'installation effectuée, il suffit d'ouvrir une console et de lancer la commande `rethinkdb` (cela va créer, depuis le répertoire où la commande est exécutée, un dossier `rethinkdb_data` contenant les données).

```
remy@MyLinux: ~  
Fichier  Édition  Affichage  Rechercher  Terminal  Aide  
remy@MyLinux:~$ rethinkdb  
Running rethinkdb 2.0.1~0jessie (GCC 4.9.1) ...  
Running on Linux 3.16.0-4-amd64 x86_64  
Loading data from directory /home/remy/rethinkdb_data  
warn: Cache size does not leave much memory for server and query overhead (available memory: 805 MB).  
warn: Cache size is very low and may impact performance.  
Listening for intracluster connections on port 29015  
Listening for client driver connections on port 28015  
Listening for administrative HTTP connections on port 8080  
Listening on addresses: 127.0.0.1, 127.0.1.1, ::1  
To fully expose RethinkDB on the network, bind to all addresses by running rethinkdb with the '--bind all' command line option.  
Server ready, "MyLinux_tbl" a13f0bf5-cb1e-468a-8387-3cfddd26ab65
```

Démarrage de RethinkDB

Découverte de l'interface web de RethinkDB

L'interface web d'administration est accessible à l'adresse <http://localhost:8080>

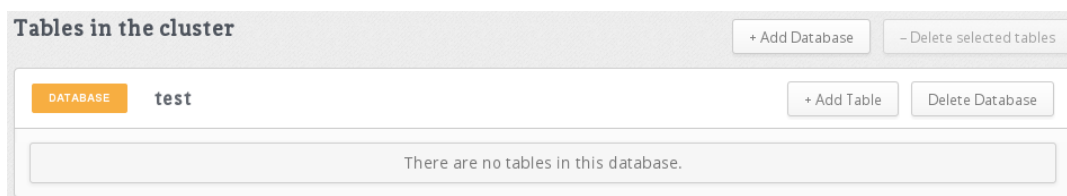


Interface Web de RethinkDB

La page principale fournit des informations telles que le nom du serveur avec lequel la connexion est établie, l'état du système (problèmes détectés, santé des serveurs et des tables, ...), les performances, ...

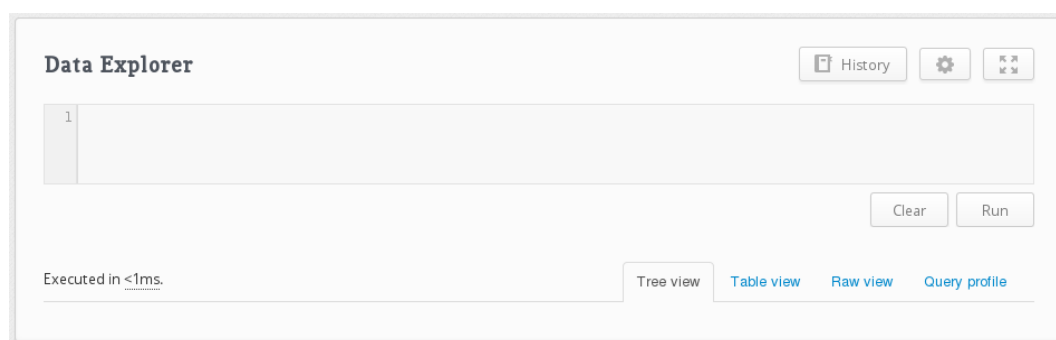
Dans la partie supérieure se situe un menu. Dans un premier temps, nous allons nous attarder sur les écrans Tables et Data Explorer.

Tables permet de voir les différentes tables ayant été créées pour stocker nos documents (une table est une collection de documents JSON). Lorsque RethinkDB vient d'être installé, il existe uniquement une base de données nommée test dépourvue de table.



Ecran Table vierge de données

Data Explorer est une interface permettant d'exécuter directement des requêtes ReQL depuis le navigateur. Le résultat peut être visualisé sous forme d'arbre, de table ou brute. Il est également possible de consulter les performances des requêtes.



Ecran Data Explorer

C'est un bon outil pour commencer à manipuler le langage ReQL et interagir avec une base de données.

Premiers tests avec le langage de requête ReQL

ReQL est le langage de requête proposé par RethinkDB. Il existe 3 drivers officiels permettant la communication entre un client et le serveur RethinkDB (JavaScript, Ruby et Python). Pour découvrir le langage ReQL, nous utiliserons le Data Explorer qui s'appuie sur la syntaxe JavaScript. L'écriture du script Python nécessitera quelques adaptations mineures dues à la syntaxe Python.

Toutes les commandes sont documentées. Voici les liens pour la syntaxe JavaScript et Python :

<http://rethinkdb.com/api/javascript>

<http://rethinkdb.com/api/python>

Afin de nous familiariser avec la syntaxe, nous effectuerons quelques opérations simples. Toutes les commandes sont chaînées avec l'opérateur `;` et commence par `r`. L'autocomplétion est très appréciable à l'usage.

Commençons et créons une base de données, une table, des documents, faisons quelques interrogations en nous appuyant sur le guide de démarrage rapide : <http://rethinkdb.com/docs/quickstart>

Voici ci-dessous les commandes et leur résultat :

Commande	<pre>//création de la BDD NFE204 r.dbCreate('NFE204')</pre>
Résultat	<pre>{ "config_changes": [{ "new_val": { "id": "18cba1f4-836c-44d9-90b8-cdb29d661108" , "name": "NFE204" } , "old_val": null }] , "dbs_created": 1 }</pre>
Interprétation	La base a été créée, elle dispose d'un id et d'un nom

Commande	<pre>//création de la table (collection) tv_shows r.db('NFE204').tableCreate('tv_shows')</pre>
Résultat	<pre>{ "config_changes": [{ "new_val": { "db": "NFE204" , "durability": "hard" , "id": "f54a2b95-e3ef-4220-9dcd-a5184f9285bc" , "name": "tv_shows" , "primary_key": "id" , "shards": [{ "primary_replica": "MyLinux_q3u" , "replicas": ["MyLinux_q3u"] }] , "write_acks": "majority" } , "old_val": null }] , "tables_created": 1 }</pre>
Interprétation	La table a été créée. Elle dispose d'un id, d'un nom, d'une clé primaire (l'id car rien n'a été précisé). D'autres informations liées au partitionnement, à la réplication, à la cohérence (nous reviendrons sur ces informations ultérieurement) sont également présentes.

Commande	<pre>//Insertion de 2 documents r.db('NFE204').table('tv_shows').insert([{ name: 'Star Trek TNG', episodes: 178 }, { name: 'Battlestar Galactica', episodes: 75 }])</pre>
Résultat	<pre>{ "deleted": 0 , "errors": 0 , "generated_keys": ["27b1f484-7fcb-4f5c-a319-3fba38c11775" , "e608celf-b90a-4b72-ba91-798f6b5b9cff"] , "inserted": 2 , "replaced": 0 , "skipped": 0 , "unchanged": 0 }</pre>
Interprétation	Les 2 documents ont été insérés et un id leur a été affecté.

Commande	<code>//Vérification du contenu de la table r.db('NFE204').table('tv_shows')</code>
Résultat	<pre>{ "episodes": 178 , "id": "27b1f484-7fcb-4f5c-a319-3fba38c11775" , "name": "Star Trek TNG" } { "episodes": 75 , "id": "e608celf-b90a-4b72-ba91-798f6b5b9cff" , "name": "Battlestar Galactica" }</pre>
Interprétation	On retrouve les 2 documents avec leur id respectif.

Voici 2 autres exemples (dont le résultat est devinable) permettant de compter ou filtrer les documents :

<code>//Nombre de documents dans la table r.db('NFE204').table('tv_shows').count()</code>
<code>//Filtre et retourne les documents ayant une valeur > 100 pour la clé episodes r.db('NFE204').table('tv_shows').filter(r.row('episodes').gt(100))</code>

Après cette rapide introduction au ReQL, nous allons étudier un cas concret d'interrogation de source de données et d'insertion de documents dans RethinkDB. Ce sera l'occasion de découvrir de nouvelles commandes ReQL.

Interrogation de l'API data.gouv.fr et insertion / mise à jour des documents dans RethinkDB

Présentation de l'API data.gouv.fr

Pour ce projet, j'ai choisi de récupérer l'ensemble des jeux de données présents sur le site <https://www.data.gouv.fr> en interrogeant l'API mise à disposition. Son fonctionnement est décrit ici : <https://www.data.gouv.fr/fr/apidoc>

Les informations récupérées sont paginées, c'est-à-dire que le contenu retourné au format JSON contient la page courante, le nombre de pages, l'url de la page précédente et de la page suivante. Cela facilitera la récupération des données.

Le service REST est accessible à l'adresse <https://www.data.gouv.fr/api/1/> à laquelle il faut ajouter la référence de la ressource, ce qui donne pour les jeux de données : <https://www.data.gouv.fr/api/1/datasets>

Des arguments peuvent être ajoutés pour indiquer le nombre de résultats par page, un ordre de tri, ...

Voyons à quoi ressemble le résultat d'une requête de type GET dans le Data Explorer. Pour cela, utilisons la très pratique commande `http` proposée par ReQL.

Commande	<code>//requête HTTP GET r.http('https://www.data.gouv.fr/api/1/datasets')</code>
Résultat (sans les documents)	<pre>{ "data": [Documents Dataset] , "next_page": "https://www.data.gouv.fr/api/1/datasets/?page=2", "page": 1 , "page_size": 20 , "previous_page": null , "total": 14739 }</pre>
Interprétation	Les informations de pagination sont des membres de l'objet récupéré et les documents sont des objets contenus dans le tableau data.

L'analyse précédente nous permet de déduire comment récupérer l'url de la page suivante et les résultats courants. Il suffit d'enchaîner la commande `http` avec la commande `getField` pour obtenir le contenu du champ voulu (`next_page` pour l'url et `data` pour les documents).

```
Ex : r.http('https://www.data.gouv.fr/api/1/datasets').getField('data')
```

L'insertion de documents en base se fait avec la commande `insert` qui prend en paramètre un document ou un tableau de documents. Il suffit donc de passer la commande précédente en paramètre.

```
Ex : r.db('data_gouv').table('datasets').insert(
      r.http('https://www.data.gouv.fr/api/1/datasets').getField('data')
    )
```

Pour ce qui est des mises à jour, cela est plus délicat. L'API `data_gouv` ne propose pas de token de synchronisation. Il est donc impossible de savoir ce qui a été modifié ou supprimé depuis la dernière synchronisation entre notre base stockée dans RethinkDB et la source `data.gouv`. Pour cet exercice, j'ai choisi de faire abstraction des suppressions et de mettre à jour les documents dans RethinkDB en me basant sur l'information `last_modified` présente sur les documents récupérés depuis `data.gouv`.

Les mises à jour sont faites avec la commande `replace` qui permet de remplacer un document par un autre.

Afin de simplifier toutes ces opérations d'insertion / mise à jour de données, j'ai décidé d'écrire un script en Python qui me permet de récupérer les données que je souhaite, quand je le souhaite.

Mise en œuvre du script de récupération et d'insertion / mise à jour des données

Le script Python que je vais présenter me permet d'insérer et mettre à jour des documents dans RethinkDB à partir de la liste des jeux de données présents sur le site `data.gouv.fr` (environ 15 000 documents au format json) en utilisant le driver RethinkDB pour Python.

Afin de pouvoir l'utiliser il est nécessaire d'installer le driver client Python pour RethinkDB. La procédure est décrite ici : <http://rethinkdb.com/docs/install-drivers/python>

Une fois le driver installé, nous pouvons vérifier que tout fonctionne en tapant les quelques lignes suivantes (il s'agit de retrouver les 2 documents que nous avons insérés lors de la découverte du langage ReQL) dans un interpréteur Python (taper la commande `python` depuis une console pour lancer l'interpréteur) :

```
import rethinkdb as r
r.connect("localhost", 28015).repl()
cursor = r.db('NFE204').table('tv_shows').run()
for doc in cursor:
    print doc
```

Nous pouvons passer à l'écriture du script. La plupart des commandes nécessaires ont déjà été vues. A celles-ci s'ajoutent les commandes :

- `expr` : permet d'effectuer de nouvelles requêtes ReQL à partir de résultats stockés dans des variables Python (résultats provenant de précédentes requêtes ReQL)
- `iso8601` : pour créer un objet temps à partir d'une chaîne de caractères au format iso 8601
- `without` : contraire de `pluck`, on y indique les informations à supprimer du résultat avant d'insérer / mettre à jour les documents

En début de script se trouvent plusieurs paramètres dont un tableau (`table_lst`) contenant les noms des ressources à partir desquelles les documents sont récupérés. L'exécution entraîne la création de la base et des

tables si elles n'existent pas. Ensuite, pour chaque ressource déclarée dans `table_lst`, les documents sont parcourus (dans l'ordre croissant lors de la première récupération puis décroissant lors des synchronisations suivantes) en ne conservant que ceux non encore présents en base ou nécessitant une mise à jour. Après un traitement (suppression d'information avant insertion), les documents sont insérés dans la base ou remplacés.

Une fois terminé, le script se présente ainsi :

```
#!/usr/bin/env python
# -*- coding:Utf-8 -*-

from collections import OrderedDict
import rethinkdb as r
from rethinkdb.errors import RqlRuntimeError

# Paramètres
db = "data_gouv"
table_lst = ["datasets", "reuses"]
# Un page_size de 1000 et un timeout de 60 (adapté au page_size) doivent permettre de
# récupérer les documents (avec un page_size trop grand, la requête échouera)
page_size = 1000
request_timeout = 60

# Connexion à RethinkDB
r.connect("localhost", 28015).repl()
# Création de la base si elle n'existe pas
if r.db_list().filter(r.row.eq(db)).count().run() == 0:
    print u"Création de la base " + db
    r.db_create(db).run()

# Pour chaque table à créer / mettre à jour
for table in table_lst:
    # Création de la table si elle n'existe pas
    if r.db(db).table_list().filter(r.row.eq(table)).count().run() == 0:
        print u"Création de la table " + table
        r.db(db).table_create(table).run()
        # Le tri se fera sur l'information created_at dans l'ordre croissant
        sort_param_dict = OrderedDict([("created_at", "created")])
    # Si la table existe
    else:
        # Le tri se fera sur last_modified pour les mises à jour et created_at pour les
        # créations, dans l'ordre décroissant
        sort_param_dict = OrderedDict([("last_modified", "-last_modified"), ("created_at", "-
        created")])
    # L'url de départ avec le nom de la ressource et le nombre de documents par page
    base_url = "https://www.data.gouv.fr/api/1/" + table + "?page_size=" + str(page_size) +
    "&page=1"
    # On boucle sur sort_param dict (paire clé valeur ordonnée en sortie)
    for operation_field, url_sort_param in sort_param_dict.items():
        # On complète l'url en indiquant l'ordre de tri
        url = base_url + "&sort=" + url_sort_param
        # Si la table n'est pas vide, on récupère l'information (last_modified ou created_at)
        # du document ayant la date la plus récente
        if r.db(db).table(table).count().run() > 0 :
            last_operation_date = r.db(db).table(table).get_field(operation_field).max().run()
            # Sinon, on fixe une date pour laquelle on est sûr que tous les documents sont plus
            # récents
        else :
            last_operation_date = "1900-01-01"
        # Tant que la boucle n'est pas interrompue par un break
        while True :
            print u"Lecture des données : " + url
            read_attempts = 1
            successful_read = False
            # Tant que la requête http échoue
            while successful_read == False:
                try:
                    # On fait un GET
                    query_results = r.http(url, timeout=request_timeout).run()
                    successful_read = True
```

```

        except RqlRuntimeError as e:
            read_attempts = read_attempts + 1
            print "Erreur: " + e.message + "\n" + str(read_attempts) + "e tentative"
        # On récupère l'information next_page
        next_page = r.expr(query_results).get_field("next_page").run()
        # On récupère les documents
        data = r.expr(query_results).get_field("data").run()
        # Si on travaille sur la date de création, on ne récupère que les documents créés
après
        if operation_field == "created_at":
            filter_info = operation_field + " > " + last_operation_date
            filtered_data =
r.expr(data).filter(r.iso8601(r.row[operation_field], default_timezone="Z").gt(r.iso8601(last_o
peration_date, default_timezone="Z"))).run()
            # Si on travaille sur la date de modification, on ne récupère que les documents
modifiés après et dont la date de création est antérieure ou égale
            elif operation_field == "last_modified":
                filter_info = operation_field + " > " + last_operation_date + " & created_at
<= " + last_operation_date
                filtered_data =
r.expr(data).filter(r.iso8601(r.row[operation_field], default_timezone="Z").gt(r.iso8601(last_o
peration_date, default_timezone="Z")) &
(r.iso8601(r.row["created_at"], default_timezone="Z").le(r.iso8601(last_operation_date, default_
timezone="Z"))))
                # On supprime quelques informations avant d'insérer / mettre à jour les documents
                projection = r.expr(filtered_data).without("community_resources",
"resources").run()
                # On compte le nombre de documents
                doc_count = r.expr(projection).count().run()
                print u"Lecture effectuée : " + "documents ayant une date " + filter_info + " : "
+ str(doc_count) + u" résultats "
                # S'il y a des documents à insérer / mettre à jour
                if doc_count > 0:
                    # Pour chaque document, on remplace l'ancien document par le nouveau
                    for doc in projection:
                        r.db(db).table(table).get(doc["id"]).replace(doc).run()
                    print u"Ecriture effectuée"
                # On met à jour la variable url avant de passer à la page suivante
                url = next_page
                # S'il n'y a plus rien à insérer / mettre à jour, on sort de la boucle et on passe
éventuellement à la collection suivante
                if not (doc_count == page_size):
                    break

```

Maintenant que nous avons un jeu de données, voyons quelques commandes avancées du langage ReQL.

Pratique du langage de requête ReQL

Nous allons poursuivre nos tests sur les tables Datasets et Reuses en écrivant dans le Data Explorer quelques requêtes ReQL de type recherche textuelle, agrégation, MapReduce, jointure et changefeeds.

Nous pouvons par exemple chercher à obtenir la liste des organisations apparaissant dans les jeux de données, puis chercher à obtenir pour l'une d'entre elles les jeux de données existants (en effet, la collection datasets est orientée jeux de données et non organisation, il faut donc parcourir les documents pour y trouver une référence à l'organisation souhaitée).

Nous pouvons ensuite chercher à connaître le nombre de jeux de données produits par chaque organisation (avec un traitement MapReduce par exemple).

Retourne la liste des organisations	<code>r.db('data_gouv').table('datasets').getField('organization')('name').distinct()</code>
Retourne les jeux de données produits par la	<code>r.db('data_gouv').table('datasets').filter(r.row('organization')('name').eq('CNIL'))</code>

CNIL	On utilise <code>eq</code> pour une recherche exacte et <code>match</code> pour une recherche basée sur les expressions régulières. Par exemple pour chercher les organisations contenant le mot <i>intérieur</i> , quelle que soit la casse, on écrira : <code>r.db('data_gouv').table('datasets').filter(r.row('organization')('name')).match('(?!i)intérieur')</code>
Retourne le nombre de jeux de données produits par chaque organisation	<code>r.db('data_gouv').table('datasets').group(r.row('organization')('name')).count()</code>
Traitement identique avec MapReduce	<code>r.db('data_gouv').table('datasets').group(r.row('organization')('name')).map(function(doc){return 1}).reduce(function(a,b){return a.add(b)})</code>

Supposons maintenant que nous souhaitions avoir des informations sur le(s) jeu(x) de données associé(s) à leur(s) réutilisation(s). Nous allons effectuer une jointure entre la table Reuses et la table Datasets.

La table Reuses contient toutes les réutilisations des jeux de données. A l'intérieur d'un document réutilisation, on trouve :

- les informations relatives à la réutilisation
- un tableau contenant quelques éléments sur les jeux de données ayant servi à la réutilisation (class, id, page, title, uri)

Afin d'accéder aux autres éléments des jeux de données (la description par exemple), il est nécessaire d'effectuer une jointure.

Voici une représentation simplifiée d'un document réutilisation et le résultat souhaité une fois la jointure effectuée :

<pre>{ "datasets": [{ "class": "Dataset", "id": "id du Dataset", "page": "url du Dataset", "title": "Titre du Dataset", "uri": "uri du Dataset" }], "description": "Description de la réutilisation", "organization": objet contenant entre autres l'id de l'organisation, "page": "url de la réutilisation", "title": "Titre de la réutilisation", "uri": "uri de la réutilisation" }</pre>	<pre>{ tableau datasets remplaçant le tableau datasets initial (il contient les objets Dataset récupérés par jointure), "description": "Description de la réutilisation", "organization": objet contenant entre autres l'id de l'organisation, "page": "url de la réutilisation", "title": "Titre de la réutilisation", "uri": "uri de la réutilisation" }</pre>
--	---

Cela s'obtient avec la requête suivante :

```
r.db('data_gouv').table('reuses').merge({"datasets":r.literal(r.row('datasets').eqJoin('id',r.db('data_gouv').table('datasets')).zip())})
```

On commence par récupérer une séquence de réutilisations puis on effectue une fusion (merge) en remplaçant (literal) le tableau datasets par un tableau datasets contenant les documents Datasets (on a passé une séquence contenant l'id des Datasets à la commande `eqJoin` en indiquant que la jointure se faisait entre cet id et l'id dans la table Datasets). La commande `zip` permet de fusionner les parties droite et gauche de la jointure.

Pour terminer avec le ReQL, intéressons-nous aux changefeeds. Ils permettent au client d'être notifié lors d'une modification sur une table, un document ou le résultat d'une requête. La notification se présente sous la forme d'un objet contenant l'ancienne et la nouvelle valeur.

Supposons que nous souhaitions avoir une notification chaque fois qu'un jeu de données fait l'objet d'une réutilisation. Les documents jeux de données disposent d'un objet `metrics` contenant différents compteurs dont le nombre de réutilisations.

Les données de la base n'étant pas mises à jour en temps réel, pour tester la commande `changes`, il faut dans un premier temps l'exécuter dans le Data Explorer pour lancer l'écoute puis exécuter le script en Python pour effectuer la mise à jour des données. Si pour certains jeux de données, la valeur du compteur de réutilisations est supérieure à la valeur précédente, alors les documents en question apparaîtront automatiquement.

Voici la requête à lancer :

```
r.db('data_gouv').table('datasets').map(
    function(doc) {
        return {'id':doc('id'), 'title':doc('title'), 'reuses':doc('metrics')('reuses')}
    }
).changes().filter(r.row('new_val')('reuses').gt(r.branch(r.row('old_val'), r.row('old_val')('reuses'), 0)))
```

Les documents subissent d'abord une transformation (seuls les champs `id`, `title` et `reuses` sont conservés). Ne sont ensuite retournés par le changefeed que ceux dont la nouvelle valeur du compteur est supérieure à l'ancienne. La commande `branch` est utilisée pour traiter le cas des nouveaux documents (si le document existe, la comparaison est faite par rapport à l'ancienne valeur, sinon par rapport à 0).

Une fois la requête lancée, nous obtenons confirmation de la souscription au flux (listening for events).

Techniques de distribution

Après cette première partie consacrée à la manipulation de données, intéressons-nous maintenant au système de partitionnement et réplication adopté par RethinkDB et réalisons quelques tests.

Etude du partitionnement et de la réplication

Les pages <http://rethinkdb.com/docs/comparison-tables>, <http://rethinkdb.com/docs/sharding-and-replication> et <http://rethinkdb.com/docs/failover> fournissent des informations intéressantes sur l'architecture de RethinkDB.

Il s'agit d'une architecture maître-esclave dans laquelle le client peut dialoguer avec chacun des nœuds.

La réplication est effectuée de manière synchrone ou asynchrone avec un paramétrage au niveau de chaque table. Les paramètres `durability` (indique si l'écriture est acquittée immédiatement ou une fois la donnée sur disque) et `write_acks` (permet d'indiquer si une écriture est acquittée lorsque la donnée a été répliquée sur un seul replica ou sur la majorité) doivent être définis via le langage ReQL.

Ex : Changement du paramètre `durability` de `hard` à `soft` au niveau de la table `Datasets`

```
r.db('rethinkdb').table('table_config').filter(r.row('db').eq('data_gouv').and(r.row('name').eq('datasets'))).update({'durability':'hard'})
```

Il existe un primary replica sur lequel sont effectuées les lectures et écritures (ce qui garantit une cohérence forte) et il est possible d'ajouter des secondary replica (les lectures sur les secondary sont possibles en ajoutant le paramètre `useOutdated` dans la requête du client).

RethinkDB utilise une méthode de partitionnement par intervalle avec comme pour la réplication, une gestion du nombre de fragments au niveau de chaque table.

En cas de défaillance, il n'existe pas de mécanisme automatisé de reprise sur panne (en cas de défaillance du nœud maître, l'élection automatique d'un nouveau nœud maître n'est pas encore gérée par RethinkDB). Le problème est aussitôt remonté au niveau de l'interface web. Les conséquences de la perte d'un serveur sont les suivantes :

- il n'est plus possible de lire ou d'écrire des données contenues dans un fragment dont le primary replica est hébergé par ce serveur (s'il existe pour ce fragment des secondary replica, une lecture est possible en mode out-of-date)
- il n'est plus possible d'écrire dans un fragment si le serveur hébergeait un secondary replica et que suite à sa défaillance le nombre de répliques restantes n'est pas suffisant pour satisfaire le paramètre `write_acks` paramétré au niveau de la table (si le nombre de répliques restantes est suffisant pour satisfaire ce paramètre, alors le système continue de fonctionner)

Mise en place d'un cluster et tests

Afin de vérifier les informations précédentes, nous allons mettre en place un cluster constitué de 3 serveurs (3 instances sur la même machine). Les tables auront le paramétrage suivant : 2 fragments, 3 répliques.

La formation du cluster est réalisée en joignant au serveur déjà lancé 2 autres serveurs RethinkDB. Pour cela, lançons 2 nouvelles fenêtres de terminal et tapons respectivement les commandes suivantes :

```
rethinkdb --port-offset 1 --directory rethinkdb_data2 --join localhost:29015
```

```
rethinkdb --port-offset 2 --directory rethinkdb_data3 --join localhost:29015
```

Au niveau de l'interface web (<http://localhost:8080> (8080 pour le 1^{er} serveur, 8081 pour le 2^e et 8082 pour le 3^e), nous pouvons voir que les 3 serveurs sont bien présents et que les données des 2 tables (Datasets et Reuses qui sont pour l'instant paramétrées avec un shard et un replica) se trouvent sur le 1^{er} serveur.

The screenshot shows the RethinkDB web interface. At the top, there's a navigation bar with links: Dashboard, Tables, Servers, Data Explorer, Logs, and a search bar. Below the navigation bar, there's a status bar with four sections: 'Connected to MyLinux_tb1', 'Issues No issues', 'Servers 3/3 connected', and 'Tables 2/2 ready'. The main content area is titled 'Servers connected to the cluster' and contains a table with three rows, each representing a server in the cluster.

Server Name	Configuration	Status
MyLinux_ark	default	2 primaries, 0 secondaries
MyLinux_i1m	default	0 primaries, 0 secondaries
MyLinux_tb1	default	0 primaries, 0 secondaries

Serveurs constituant le cluster

Déclarons maintenant 3 replicas et 2 shards pour chaque table. L'opération se fait simplement depuis l'interface web :

Déclaration du nombre de fragments et répliques pour la table Datasets puis Reuses

DATABASE data_gouv		+ Add Table	Delete Database
☐	datasets	1 shard, 1 replica	Ready 1/1
☐	reuses	1 shard, 1 replica	Ready 1/1

Etat des tables avant modification du nombre de fragments et répliques

DATABASE data_gouv		+ Add Table	Delete Database
☐	datasets	2 shards, 6 replicas	Ready 6/6
☐	reuses	2 shards, 6 replicas	Ready 6/6

Etat des tables après modification du nombre de fragments et répliques

Vérifions maintenant que le système fonctionne comme décrit précédemment.

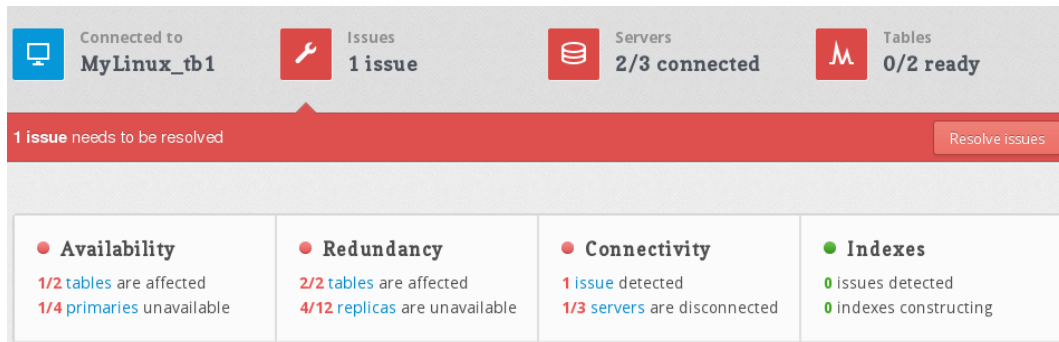
Commençons par effectuer une lecture d'un document depuis le Data Explorer à partir de l'interface web de chaque serveur :

```
r.db('data_gouv').table('datasets').get('53699d0ea3a729239d205b2e')
```

Sur chaque serveur, nous obtenons bien en retour le document demandé. La lecture a été dirigée vers le fragment primaire contenant le document. Malheureusement, je n'ai trouvé aucun moyen de l'identifier.

Simulons maintenant une panne en arrêtant le serveur RethinkDB sur lequel se trouve le fragment primaire contenant notre document puis tentons à nouveau de lire ce dernier. Ne sachant pas de quel serveur il s'agit, nous essayons successivement avec les 2 primary définis pour la table Datasets (MyLinux_ark et MyLinux_i1m).

Une fois le serveur arrêté, nous regardons ce qui se passe au niveau de l'interface web d'un des 2 serveurs restants. Nous voyons que le système nous alerte des dysfonctionnements.



Etat du cluster suite à l'indisponibilité d'un des serveurs

Le serveur MyLinux_i1m n'étant plus vu comme opérationnel par le reste des membres du cluster, il disparaît des serveurs connectés (2/3). Il en est de même pour la disponibilité et la redondance (1 table affectée par la perte d'un fragment primaire et les 2 tables affectées par la perte de 4 fragments au total (1 primaire et 3 secondaires)).

Comme prévu, la lecture et l'écriture ne sont pas possibles et le système nous renvoie le message d'erreur suivant : Cannot perform read (respectivement write) : Primary replica for shard ["53699c\x3BM\x7D\x06", +inf) not available (server 5eec7f96-97e1-4510-a254-1617300b77ce is not ready) (on voit au passage qu'il s'agit bien d'un partitionnement par intervalle).

L'ajout de l'argument useOutdated permet d'effectuer une lecture sur les fragments secondaires qui peuvent contenir des données qui ne sont pas à jour (cohérence à terme) :

```
r.db('data_gouv').table('datasets',{useOutdated:true}).get('53699d0ea3a729239d205b2e')
```

Concernant le cas de l'écriture où le serveur en panne ne contenait qu'un fragment secondaire et où le nombre de serveurs restants suffisent à satisfaire le paramètre write_acks, vérifions en relançant le primary et en arrêtant un secondary puis le second.

Dans le premier cas, il reste le primary et un secondary, le paramètre write_acks est respecté, l'écriture fonctionne (le système indique que le document a été remplacé).

Dans le second cas, il ne reste que le primary, le paramètre write_acks n'est pas respecté, l'écriture échoue avec le message suivant : Cannot perform write: primary replica error: The number of available replicas is smaller than the requested number of replicas to acknowledge a write.

Comme nous l'avons déjà évoqué, il n'existe pas de basculement automatique suite à un problème. Il faut attendre que le serveur soit de nouveau opérationnel pour que le système retrouve son état ou il faut le supprimer et reconfigurer la partie sharding et replication des tables affectées.

Interfaçage avec le moteur de recherche Elasticsearch

ElasticSearch (<http://rethinkdb.com/docs/elasticsearch>), que nous avons déjà rencontré pendant le cours, va nous permettre de stocker et indexer certaines informations issues de notre base de données documentaire.

La finalité étant d'effectuer des requêtes pour obtenir des tableaux de bord analytiques dans Kibana (outil sur lequel nous viendrons juste après), nous allons partiellement déclarer un schéma afin de pouvoir exploiter correctement les données de l'index. En effet, par défaut, Elasticsearch effectue un mapping dynamique qui ne correspond pas nécessairement à nos attentes (tokenisation non souhaitée d'une chaîne de caractères par exemple).

Création d'un index avec mapping

Commençons par créer un index dans Elasticsearch et spécifions une partie du mapping. Il n'est pas question ici de faire un mapping élaboré (choix des informations à conserver, stockage ou indexation de l'information, utilisation d'analyseur, ...) mais simplement de donner un exemple de mapping. On souhaite indiquer au système de ne pas appliquer d'analyse sur les champs de type texte.

Pour cela, nous créons un fichier mapping.json dans lequel nous plaçons la structure suivante :

```
{
  "mappings": {
    "_default_": {
      "dynamic_templates": [
        {
          "string_fields": {
            "match": "*",
            "match_mapping_type": "string",
            "mapping": {
              "type": "string",
              "index": "not_analyzed"
            }
          }
        }
      ]
    }
  }
}
```

Cette déclaration permet de conserver un mapping dynamique en modifiant le comportement par défaut. Le template string_fields sera appliqué à tous les nouveaux champs de type texte rencontrés et ces derniers ne seront pas analysés (plus d'infos ici : <https://www.elastic.co/guide/en/elasticsearch/guide/current/custom-dynamic-mapping.html>).

L'index peut ensuite être créé via un PUT. Nous indiquons le nom de l'index et le chemin du fichier contenant le mapping :

```
curl -X PUT localhost:9200/data_gouv -d @mapping.json
```

Suite à cette commande, l'index apparaît dans l'interface web et il est possible de regarder les métadonnées de l'index pour vérifier la prise en compte du mapping.

Création d'une river

Afin que les documents insérés dans notre base de données documentaire soit automatiquement synchronisés avec l'index, nous allons maintenant déclarer la river RethinkDB.

Il faut tout d'abord installer le plugin en exécutant la commande fournie dans la documentation (<http://rethinkdb.com/docs/elasticsearch>) depuis un terminal :

```
plugin --install river-rethinkdb --url http://goo.gl/JmMwTf
```


Une fois le plugin installé et Elasticsearch relancé (pour la prise en compte), il reste à déclarer la river. Comme pour le mapping, nous plaçons les informations de configuration dans un fichier river.json :

```
{
  "type": "rethinkdb",
  "rethinkdb": {
    "host": "localhost",
    "port": 28015,
    "databases": {
      "data_gouv": {
        "datasets": { "backfill": true },
        "reuses": { "backfill": true }
      }
    }
  }
}
```

Puis nous créons la river via un PUT :

```
curl -X PUT localhost:9200/_river/rethinkdb/_meta -d @river.json
```

Suite à cette commande, les documents sont progressivement insérés dans l'index et si nous regardons les métadonnées, nous constatons qu'un mapping dynamique a été effectué sur chacun des champs et que les champs de type texte sont bien non analysés.

Réalisation d'un tableau de bord avec l'outil d'analyse Kibana

Maintenant que l'index Elasticsearch contient des informations, nous allons effectuer des requêtes et réaliser un tableau de bord avec Kibana (documentation : <https://www.elastic.co/guide/en/kibana/current/index.html>).

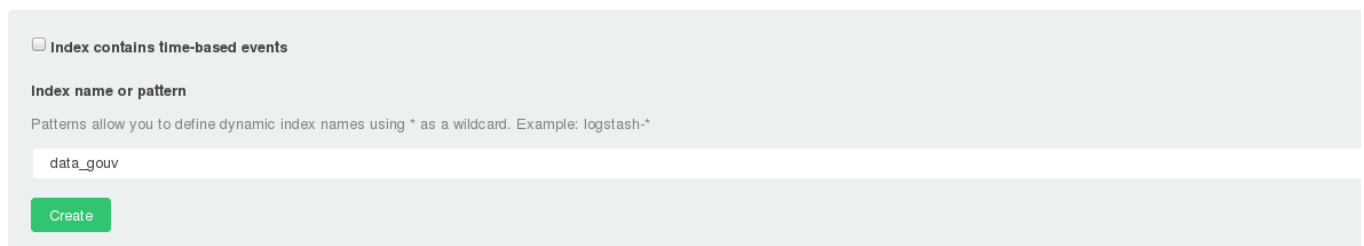
L'installation se fait aisément, il faut décompresser l'archive (<https://www.elastic.co/downloads/kibana>) dans un dossier puis lancer le programme. Il est ensuite possible d'accéder à l'interface web : <http://localhost:5601>

Création d'un index pattern

Lors du premier accès à Kibana, il nous est demandé de définir un index pattern correspondant à l'index Elasticsearch pour lequel on souhaite exploiter les données. Il est également possible de préciser quel champ de type time sera utilisé par le filtre temporel. Nous indiquons le nom de notre index et ignorons la partie temporelle car cela ne présente pas vraiment d'intérêt avec nos données (cela est plus pertinent avec des logs par exemple) :

Configure an index pattern

In order to use Kibana you must configure at least one index pattern. Index patterns are used to identify the Elasticsearch index to run search and analytics against. They are also used to configure fields.



☐ Index contains time-based events

Index name or pattern

Patterns allow you to define dynamic index names using * as a wildcard. Example: logstash-*

data_gouv

Create

Configuration de l'index pattern

On voit alors apparaître la liste des champs présents dans l'index et des informations les concernant (type, indexés ?, analysés ?).

Présentation de l'interface

Un fois l'index pattern créé, il devient possible d'effectuer des recherches, créer des vues, et des tableaux de bord. Le menu propose 4 actions :

- Discover : Choix de l'index à interroger (un seul dans notre cas), exploration des données, sélection des champs, filtre sur les résultats, recherches en utilisant le langage de requête d'ElasticSearch (Lucene ou DSL), enregistrement de la recherche ou ouverture d'une recherche existante
- Visualize : Création de nouvelles vues en choisissant un modèle visuel, en se basant sur une vue ou une recherche existante ou à partir d'une nouvelle recherche. L'éditeur, décomposé en 3 parties, permet d'agréger les données et choisir les options de présentation, effectuer des recherches, visualiser le résultat
- Dashboard : Disposition des vues ou des recherches enregistrées sur un tableau de bord et recherches sur les données présentées (toutes les vues constituant le tableau de bord sont affectées par cette recherche)
- Settings : Accès à différents paramètres de configuration

N.B. : Toutes les opérations effectuées dans Kibana (création de l'index pattern, des recherches, vues, tableaux de bord, modification des paramètres, ...) sont stockées dans ElasticSearch (index portant le nom .kibana).

Réalisation d'un tableau de bord à partir des données

Avant de se lancer dans la réalisation de notre tableau de bord, il convient de définir les indicateurs à mettre en place. Les données récupérées ne sont malheureusement pas très pertinentes d'un point de vue analytique mais suffisent à avoir un aperçu des possibilités offertes par Kibana. Pour notre tableau de bord, seront donc mises en places, les vues suivantes :

- Nombre de Datasets : vue de type Metric indiquant le nombre de documents provenant de la table Datasets
- Nombre de Reuses : identique au précédent mais avec la table Reuses
- Répartition des Datasets par type de licence : vue de type Pie Chart montrant la quantité de documents par type de licence et leur répartition (%) (documents groupés sur les valeurs du champ licence)
- Top 10 des Datasets les plus réutilisés : vue de type Table présentant les 10 Datasets les plus réutilisés (titres des 10 documents ayant le champ reuses avec la plus grosse valeur (max))
- Top 10 des organisations publiant le plus de Datasets : vue de type Vertical bar chart présentant la quantité de documents par organisation (documents groupés sur les valeurs du champ organisation.name, seuls les 10 plus gros résultats sont retournés)
- Quantité de documents ajoutés par mois : vue de type Vertical bar chart où les données sont groupées par mois sur le champ created_at. On définit un second niveau d'agrégation sur le champ _type (indique si l'information provient de la table Datasets ou Reuses)
- Top 10 des tags par type : vue de type Pie chart avec 2 niveaux d'agrégation également, un premier cercle avec la répartition des documents par type et un 2^e indiquant pour chaque type le top 10 des tags y apparaissant le plus

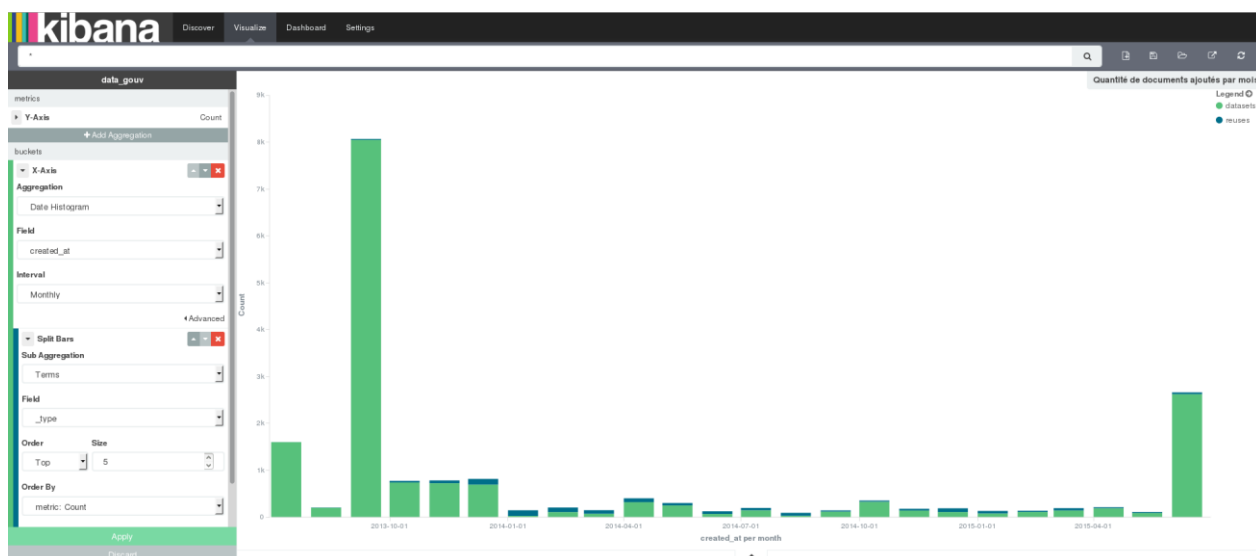
A noter que pour certaines vues où il est précisé dans le titre si les données traitées proviennent de Datasets ou Reuses, il est nécessaire d'appliquer, dans la barre de recherche, un filtre sur le champ _type. Sans quoi, on se retrouve avec des données agrégées sans lien.

Voici la procédure détaillée pour la vue représentant la quantité de documents ajoutés par mois.

Dans Visualize, nous choisissons Vertical bar chart, from a new search, puis nous indiquons pour l'axe Y une agrégation de type Count et pour l'axe X une agrégation par date (intervalle en mois) sur le champ created_at. Nous ajoutons ensuite un niveau supplémentaire avec un éclatement Split bar et une agrégation de type Terms sur le champ _type sur le champ _type.

Détail du paramétrage de la vue « Quantité de documents ajoutés par mois »

Nous obtenons le résultat suivant :



Vue « Quantité de documents ajoutés par mois »

Il ne reste plus qu'à sauvegarder la vue pour pouvoir l'appeler lors de la mise en place du tableau de bord. Une fois toutes les vues créées et insérées dans le tableau de bord, ce dernier se présente comme suit :

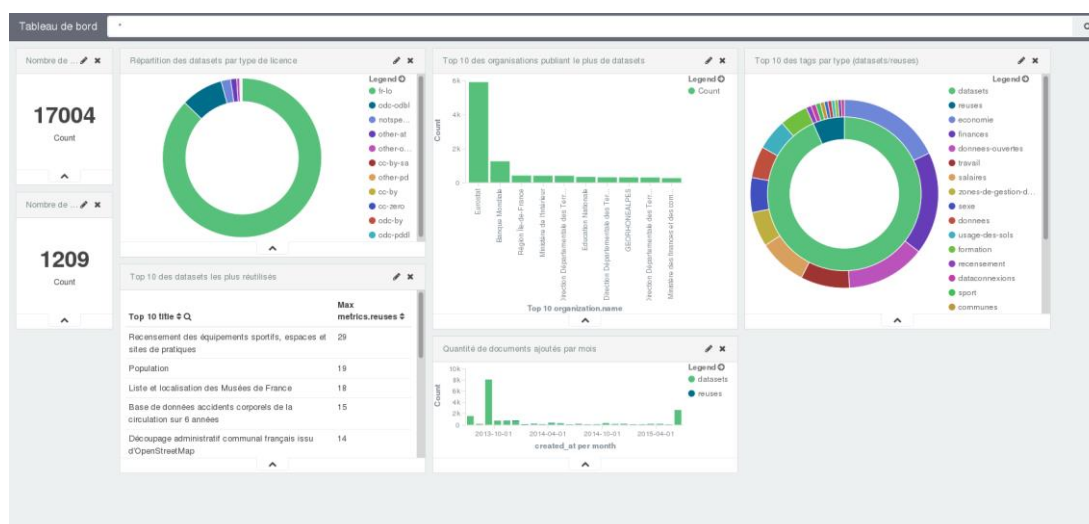
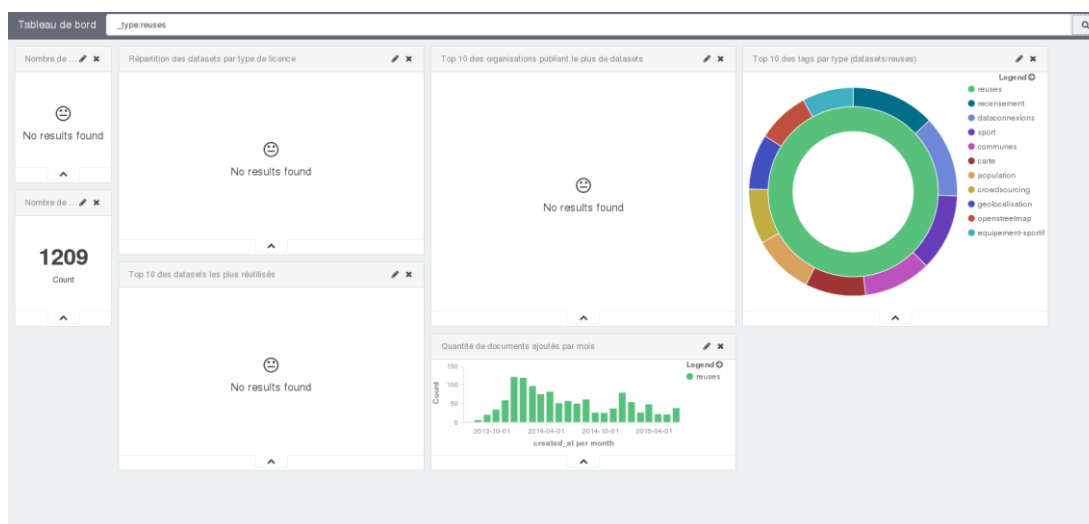


Tableau de bord regroupant les vues créées

Ici aussi, il est possible d'appliquer des recherches, comme évoqué dans la présentation de l'interface.



Le même tableau de bord avec un filtre sur `_type:reuses`

Conclusion

Le cours NFE204 (Bases de données documentaires et distribuées) nous a donné, à travers un enseignement mêlant théorie et pratique, les bases nécessaires à la réalisation de ce projet. Nous avons pu aborder des sujets tels que la modélisation de données peu structurées, la recherche d'informations, les systèmes NoSQL et des problématiques liées aux systèmes distribués comme le passage à l'échelle, la reprise sur panne, le stockage, le partitionnement et la réplication.

Les connaissances acquises m'ont permis de réaliser cette étude, au cours de laquelle j'ai pu examiner le système NoSQL RethinkDB, en particulier son langage ReQL et son système de partitionnement et réplication. Cela a également été l'occasion de revoir et approfondir certains aspects comme la récupération de données depuis un service REST, la création d'une river, la mise en place d'un schéma (vu en cours avec Solr uniquement) au niveau d'ElasticSearch et la découverte de l'outil d'analyse Kibana.

Dans le cours, tous les aspects des systèmes présentés n'ont pas été abordés en intégralité, et c'est également le cas pour mon étude. En effet, il existe de nombreux systèmes sur le marché, chacun possède ses propres spécificités et il est difficile de les étudier de bout en bout.

Pour aller plus loin, nous pourrions envisager, en situation réelle, avec d'autres sources de données, et en fonction des besoins, d'approfondir des aspects tels que le choix du système NoSQL à déployer en effectuant des comparatifs, la mise en place d'une indexation efficace en spécifiant un schéma (ou mapping) adéquat, la création de tableaux de bord pour procéder à de l'analyse temps réel, ...